

TP3

Perceptrones

TP3 — Perceptrones
Implementación desde cero con NumPy

Sistemas de Inteligencia Artificial · ITBA · 1Q 2026

Bloque 1 / 4

Introducción + Engine + Validación

Qué pide el TP

Tres ejercicios + validación, todo desde cero en NumPy.

Ej 0	Validación: AND (step), XOR (MLP), lineal y tanh
Ej 1	Detección de fraude por <i>knowledge distillation</i>
Ej 2	Clasificación de dígitos manuscritos (MLP, 10 clases)
Ej 3	Lo mismo que Ej 2, target: accuracy $\geq 98\%$

Restricción explícita del enunciado: sin sklearn estimators, sin PyTorch ni TensorFlow como capas. Solo NumPy para matrices, Pandas para datos, Matplotlib para plots.

Filosofía: un engine, muchos configs

Decisión arquitectural:

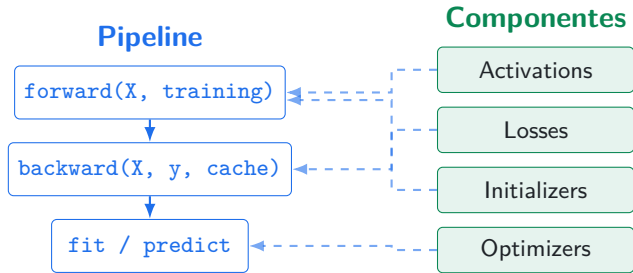
Módulo `mlp/` reutilizable + configs JSON.

Beneficios:

- Una codebase, 78 tests verde
- Cambiar de Ej0 a Ej3 = cambiar de JSON
- Pack C del Ej3 = 3 líneas en `regularization`

```
mlp/  
<- engine (NumPy)  
ejercicio0/  
<- AND, XOR, validacion  
ejercicio1/  
<- fraud (Tomas, scripts aparte)  
ejercicio2/           <- digitos  
ejercicio3/  
<- digitos >= 98%  
docs/  
<- specs, plans, guias
```

Anatomía del engine mlp/



Activaciones

Losses

Optimizadores

sigmoid, tanh, relu, identity, softmax

MSE, BCE, cross_entropy

SGD, Momentum, Adam

Bias trick: cada $W^{(l)}$ tiene shape $(n_{l+1}, n_l + 1)$, columna 0 es bias.

Ej 0 — AND con perceptrón escalón

Representación bipolar (apunte): entradas $\{-1, +1\}$, salida esperada $[-1, -1, -1, +1]$.

Update con función signo:

$$\Delta w = \eta (z - O) x \quad \text{con } O = \text{sign}(w \cdot x)$$

Test: converge a error = 0 en pocas epochs.
Implementación en
`ejercicio0/step_perceptron.py`.

```
{  
  "loss": "mse",  
  "optimizer": {"name": "sgd", "lr":  
                :0.1},  
  "activations": ["step"],  
  "epochs": 100  
}
```

Validación pre-engine: si AND no converge, hay un bug en la pipeline.

Ej 0 — XOR como red de seguridad

XOR no es separable linealmente. Necesita capa oculta no-lineal.

Probamos dos arquitecturas:

- $[2, 2, 1]$ — la mínima que resuelve XOR (configs/xor_2_2_1.json)
- $[2, 3, 2, 1]$ — con capa extra para validar arquitecturas profundas

XOR es la red de seguridad del engine.

Durante el desarrollo encontramos 4 bugs reales que se manifestaban acá antes que en dígitos:

- `predict()` devolvía $\{0, 1\}$ en vez de $\{-1, +1\}$ para tanh-binario.
- `identity` activation aliasaba su input.
- `numerical_grad` con int dtype daba garbage silencioso.
- Mean/std del `run_summary` perdía strings por dict-spread overwrite.

Bloque 2 / 4

Ej 1 — Knowledge Distillation

Ej 1 — El problema

Knowledge distillation: replicar la salida de un BigModel pre-entrenado con un TinyModel (perceptrón simple).

Datos: `fraud_dataset.csv`

9 features (timestamp, amount, session...)

Target de entrenamiento:

`big_model_fraud_probability` (continuo en $[0, 1]$)

Ground truth de evaluación:

`flagged_fraud` (binaria)

Regla del enunciado:
`flagged_fraud` *nunca* se usa para entrenar. Solo para métricas operativas (precision/recall/F1) post-hoc.

Salida en $[0, 1]$ $\Rightarrow$ activación sigmoide.

Ej 1 — Setup técnico

Pipeline: K-fold estratificado por `flagged_fraud`, normalización z-score *fit-on-train-only* (sin leakage), online learning sample-by-sample.

Features excluidas (3 de 12 columnas): `timestamp`, `device_screen_resolution`, `time_since_last_login_s`.

```
// ejercicio1/nonlinear_perceptron/configs/baseline.json
{
  "target_col": "big_model_fraud_probability",
  "eval_col":   "flagged_fraud",
  "exclude_features": ["timestamp", "device_screen_resolution",
                      "time_since_last_login_s"],
  "k_folds": 5,
  "stratify": true,
  "training": {"learning_rate": 0.0001, "epochs": 500, "epsilon": 1e-5},
  "evaluation": {"threshold": 0.5},
  "normalization": "zscore"
}
```

Ej 1 — Lineal vs No-Lineal

Lineal (Adaline)

$$O = w \cdot x$$

$$\Delta w = \eta (z - O) x$$

No-Lineal (Sigmoide)

$$O = \sigma(w \cdot x)$$

$$\Delta w = \eta (z - O) O(1 - O) x$$

El factor $O(1 - O)$ modula el gradiente: se achica en zonas saturadas (cerca de 0 o 1), es máximo cerca de 0.5.

Modelo	MSE test (mean $\pm$ std)
Lineal	0,0266 $\pm$ 0,0008
No-Lineal	0,0110 $\pm$ 0,0004 (−59 %)

Ej 1 — Por qué la sigmoide gana

El target está acotado en $[0, 1]$. La sigmoide mapea naturalmente al rango del target. El lineal puede producir valores fuera del rango y el MSE lo penaliza fuerte.

Implementación con estabilidad numérica: la sigmoide del código de Tomás usa máscara para evitar overflow en `exp`:

```
def sigmoid(x):  
    out = np.empty_like(x)  
    pos = x >= 0  
    out[pos] = 1.0 / (1.0 + np.exp(-x[pos]))  
    exp_x = np.exp(x[~pos])  
    out[~pos] = exp_x / (1.0 + exp_x)  
    return out
```

Ej 1 — El threshold default es engañoso

Con threshold = 0.5: precision = 0.33, recall = 1.00.

Casi todo se clasifica como fraude. **Pero el modelo no es malo** — el threshold está mal.

Las salidas sigmoide se concentran: la mediana de los scores de fraude está en **0.99**. Los no-fraudes tampoco bajan a 0.05; bajan a 0.4–0.6. El modelo aprendió ranking, no calibración.

Solución: barrer el threshold de 0.00 a 1.00, paso 0.01.

Ej 1 — Threshold sweep

Threshold	Precision	Recall	F1	Accuracy
0.50	0.333	1.000	0.499	0.768
0.80	0.751	0.952	0.840	0.958
0.85	0.812	0.893	0.850	0.964
0.89	0.886	0.861	0.873	0.971
0.95	0.949	0.746	0.835	0.966
0.99	1.000	0.527	0.690	0.945

AUC-ROC: no-lineal 0,9921 vs lineal 0,9720.

Recomendación: threshold = 0,89 si se prioriza F1; $\geq 0,95$ si los falsos positivos son costosos. Decisión de negocio sobre el costo asimétrico FN/FP.

Bloque 3 / 4

Ej 2 — Clasificación de dígitos

Ej 2 — El problema

Clasificación multiclase de dígitos manuscritos (0–9).

Datos:

- Train: `digits.csv` (12.449 muestras)
- Test: `digits_test.csv` (2.498)

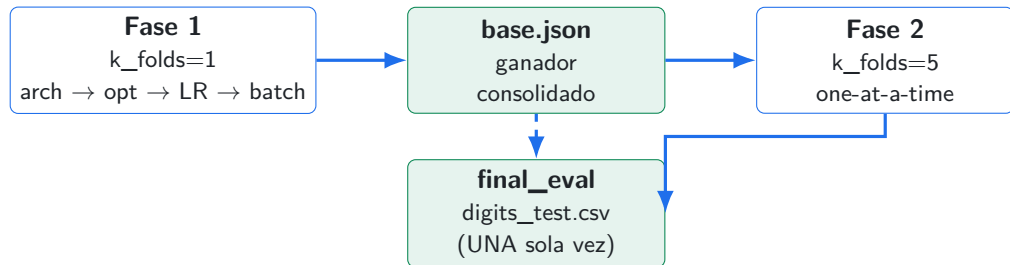
Cada muestra: vector de 784 floats $\in [0, 1]$
(28×28 flatten).

Arquitectura final:

- Input: 784
- Hidden 1: 100 (ReLU)
- Hidden 2: 50 (ReLU)
- Output: 10 (Softmax)

Regla: `digits_test.csv` se evalúa una sola vez, al final. No se toca durante búsqueda de hiperparámetros.

Ej 2 — Workflow disciplinado



Reduce $\mathcal{O}(n^4)$ a $\mathcal{O}(4n)$. Sweeps secuenciales en Fase 1; one-at-a-time en Fase 2 valida la elección con $K=5$.

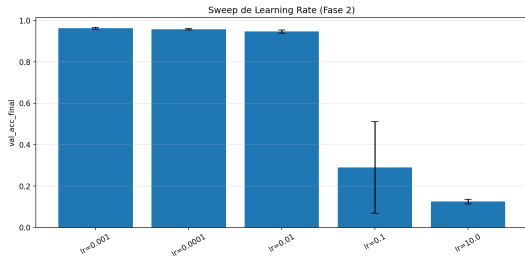
Ej 2 — Ganadores de Fase 1

Sub-fase	Ganador	val_acc	Comentario
1.1 Arquitectura	[784, 100, 50, 10]	0.9674	Empate técnico, parsimonia
1.2 Optimizador	adam (lr=0.001)	0.9674	SGD no convergió en 50 ep
1.3 Learning rate	0.001	0.9674	0.0001 lento, $\geq 0,005$ overshoot
1.4 Batch size	16	0.9674	best_ep=3, el más rápido

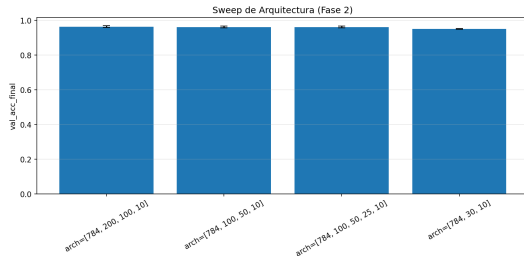
`base.json` consolidado, K-fold=5:

$\text{val_acc} = \mathbf{0,9622 \pm 0,0043}$

Ej 2 — Fase 2 valida la elección



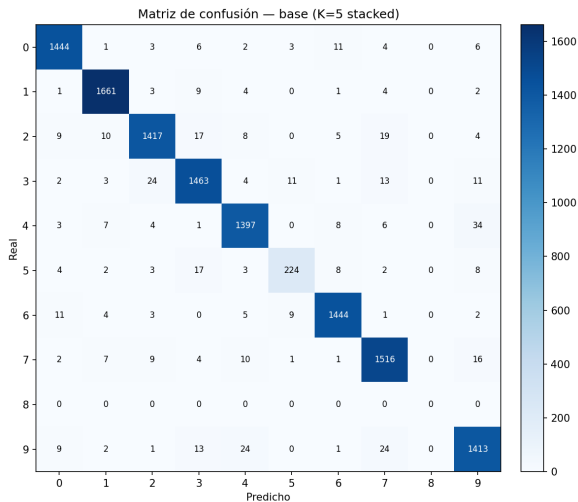
Sweep LR — `lr=10` es accuracy random



Sweep arquitectura — todas en empate

Solo el sweep de LR muestra rangos catastróficos. Arch, opt e init quedan en empate técnico — **Fase 2 valida más que descubre.**

Ej 2 — Curvas y matriz del base (val K=5)



Diagonal limpia para 9 de 10 clases.

La clase 8 colapsa: precision = recall = 0.

El modelo no la predice nunca en val.
Indicador de algo raro en la distribución del train (sub-representación de la clase 8 o ambigüedad en los features).

val K-fold=5: **96.22 %**



test (digits_test.csv): **86.30 %**

Drop de 10 puntos porcentuales.

El modelo está bien — los datos no se parecen.

TP3 — Perceptrones
La matriz de confusión sobre test confirma: las confusiones se reparten por todo el grid, no es una clase específica. Patrón de **distribution shift**, no de underfitting.

Ej 2 → Ej 3 — La hipótesis

Si el problema es shift entre `digits.csv` y `digits_test.csv`, no falta de capacidad:

⇒ el primer movimiento debería ser **más datos**, no un modelo más grande.

Lo que provee el enunciado del Ej 3: `more_digits.csv` (15.742 muestras adicionales).

Lo que el engine ya soporta: `extra_csv_paths` en el config concatena CSVs antes del split.

→ **cero cambios al modelo, solo cambia un campo del JSON.**

Bloque 4 / 4

Ej 3 — Pack C y conclusiones

Ej 3 — Plan

Estrategia secuencial:

1. Concatenar `more_digits.csv`. Si $\geq 0,98$, listo.
2. Activar Pack C (regularización del enunciado): L2, dropout, augmentation, lr_schedule.
3. Si nada llega, escalar capacidad (`arch_wider`).

Pack C en el engine: vía campo `regularization` del config.

```
"regularization": {  
    "l2": 1e-4,  
    "dropout": 0.2,  
    "lr_schedule": {"type": "step", "decay": 0.5, "every": 10},  
    "augmentation": {"type": "gaussian_noise", "sigma": 0.05}  
}
```

Implementamos dropout, augmentation y lr_schedule en `mlp/network.py`; tests verde 78/78.

Ej 3 — El movimiento dominó

val $K=5$: 96,22 % $\rightarrow$ 97,06 %

test: 86,30 % $\rightarrow$ 96,36 %

+10 puntos porcentuales en test.
Cero cambios al modelo. Solo más datos.

Macro F1: 0,857 $\rightarrow$ 0,959.

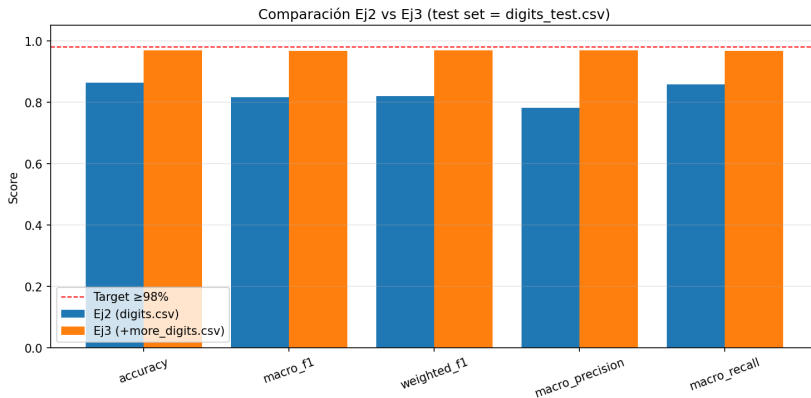
La clase 8 que estaba colapsada en Ejercicio 2 ahora se predice correctamente.
Confirma la hipótesis del shift.

Ej 3 — Iteración Pack C

Config	val K=5	test	Δ vs base_extra
base_extra_data	0.9706	0.9636	— (referencia)
+ L2 (1e-4)	0.9758	0.9656	+0,20 pp
+ dropout (p=0.2)	0.9750	0.9628	−0,08 pp
+ L2 + dropout	0.9772	0.9604	−0,32 pp
+ L2 + aug ($\sigma = 0,05$)	0.9760	0.9688	+0,52 pp
+ L2 + aug + dropout	0.9768	0.9656	+0,20 pp
+ wider + L2 + aug	0.9777	0.9640	+0,04 pp

Ganador: L2 + augmentation gaussiana ($\sigma = 0,05$) → **96.88 % en test.**

Ej 3 — Comparación visual con Ej 2



Línea roja: target $\geq 98\%$.

Ej 3 — Qué movió la aguja

Lo que ayudó:

- **Más datos: +10pp.** El movimiento dominante.
- **L2: +0,20pp** consistente.
- **Aug gaussiana: +0,32pp** *adicional* sobre L2 (sinergia).

Lo que no ayudó:

- **Dropout:** gana en val (+0,44pp), pierde en test (−0,08pp). Reduce varianza específica al fold, no al shift.
- **wider [784, 200, 100, 10]:** mejor en val, peor en test. Más capacidad memoriza, no generaliza. **Descarta que falte capacidad.**
- **Combinar todo:** L2+dropout+aug peor que L2+aug. La regularización compite consigo misma.

Ej 3 — Por qué no llegamos al 98 %

Mejor: 96.88 %. Faltan 1.12pp.

Tres hipótesis ordenadas por probabilidad:

1. **Augmentation** insuficiente.

Gaussian noise es *isotrópico* (independiente por píxel). El shift parece ser de *estilo de escritura*: rotación, traslación, grosor de trazo. Pide augmentation **geométrica**, no por noise. No la implementamos en este TP.

2. **Sub-representación en val interno del final_eval.**

El final_eval usa val 10 % random para early stopping. Si los digits “difíciles” del test no aparecen ahí, la red para antes de tiempo.

3. **Capacidad**: improbable, *wider* sobreajustó.

Próximo paso: augmentation por rotación/traslación pequeñas, mantener L2 + σ bajo de gaussian noise como rutina, reconsiderar arquitectura solo después.

Conclusiones del TP

Tres ideas para llevarse:

1. Una buena pipeline gana más que tunear hiperparámetros: el +10pp del Ej 3 vino de *datos*, no de modelo.
2. Los empates técnicos son mucho más comunes que los wins claros. Elegir por **parsimonia**, no por 0.05pp en val.
3. “Mejor en val” $\neq$ “mejor en test” cuando hay shift. Dropout fue el ejemplo: ganador en val K-fold, perdedor en test.

Item	Status
Ej 0 — step AND, MLP XOR, lineal y tanh	✓
Ej 1 — Knowledge distillation + threshold sweep	✓
Ej 2 — MLP from-scratch + sweeps + final_eval	✓
Ej 3 — more_digits.csv + análisis Pack C	✓
Ej 3 — target $\geq 98\%$	✗ (96.88 %)

Gracias.

Preguntas.

`github.com/pi-na/SIA-TP3 · branch`

`Companion: docs/informe_tp3.pdf · docs/guia_compañeros.md`